

Course Name : Fundamentals of Computer and Programming

Project Title : Checking Strong Connectivity in a Directed Graph

Student Name : Saleh Zabihpazhouh

Student ID : 40334184

Submission Date : 1403/11/11

1. Introduction

1.1. Problem Statement

- We have a graph $G = (V, E)$. We need to write a program in Python to say that it is possible to go from any vertex to any other vertex in this graph.
- Investigating strong connectivity in a directed graph is very important to understand the structure and behavior of the graph. It can go from any vertex to any other vertex and this is very useful. In fact, it can help some applications such as: network design, social network analysis, web crawling and search engines.

1.2. Objective

- 1. Create a strong connectivity checker.
 - 2. Use an efficient algorithms.
 - 3. Graph representation and traversal.
 - 4. Handle different graph size.
 - 5. Provide clear result.
 - 6. Learn and explain.
-

2. Methodology

2.1. Algorithms Description

- I use Kosaraju's Algorithm. First use Depth-First-Search (DFS) if all vertex were gone now we transpose the graph and again use Depth-First-Search (DFS) and again if all vertex were gone

can say the graph is strong connected but if not all the vertex are gone for the first DFS and second DFS can say the graph isn't strong connect.

2.2. Implementation Details

- Use the Python language and use PyCharm text editor and File Handling and don't use any libraries.
 - Input File Format, Graph Type, Vertex Indexing, Edge Weights, Strong Connectivity.
-

3. Function Documentation

1. create_graph(filename) :

Purpose:

First open filename and assign it to a variable like f

And read directed graph from file and constructs and adjacency list representation using only available edges.

Parameters:

Filename is a string type and it the name of the input file containing graph data.

Return Value:

Return list of list where each index represents a vertex and the list at that index contains all is outgoing neighbors.

Logic:

1. Reads the number of vertex from the first line of the file.
2. Initializes an adjacency list with empty list for each vertex.
3. Iterates through each line extracting u,v, and w.
4. If w = 1 adds an edge from u -> v to the adjacency list.

2. reverse_graph(graph):

Purpose:

Create a transpose version of the graph by reversing all edge.

Parameters:

Graph(list[list[int]]) -> the adjacency list of the original graph.

Return Value:

A new adjacency list where each edge $u \rightarrow v$ is reversed $v \rightarrow u$.

Logic:

1. Creates an empty adjacency list(re_graph).
2. Iterates through each vertex and its outgoing edges.
3. Adds a reversed edge $v \rightarrow u$ to re_graph.

3. dfs(graph,node,visited):

Purpose:

Perform Depth-First Search (DFS) from given vertex to mark all reachable nodes.

Parameters:

Graph(list[list[int]]): The adjacency list representation of the graph.

Node(int): The current vertex being visited.

Visited(list[bool]): A list tracking visited nodes (True if visited, False otherwise).

Return Value:

Modifies visited in place to mark all reachable nodes.

Logic:

1. Marks the current node as visited.
2. Recursively calls dfs for all unvisited neighbor of node.

4. Check_CSC(graph):

Purpose:

Checks if the given graph is strongly connected using Kosaraju's Algorithms.

Parameters:

Graph(list[list[int]]): the adjacency list representation of the graph.

Return Value:

True -> if the graph strongly connected.

False -> if the graph isn't strongly connected.

Logic:

1. Perform a DFS from vertex 0 and check if all nodes are visited.

- If any node is unreachable return False
2. Transpose the graph (reverse all edges).
 3. Perform DFS again on the transpose graph from vertex 0.
 - If all nodes are reachable the graph is strongly connected.
 - Otherwise return False.
-

4. Input and Output

4.1. Input Format

- Input format is a file.txt and in this file define a directed graph and can read graph structures.

4.2. Output Format

- Output is bool if graph is strongly connected return True otherwise return False.

5. Conclusion

In this project, we successfully implemented a Depth-First Search (DFS) based algorithm to determine whether a directed graph is strongly connected. Using Kosaraju's Algorithm, we efficiently checked if every node in the graph is reachable from every other node.

The project involved:

- Reading a directed graph from text file.
- Constructing an adjacency list while considering only valid edge ($w = 1$).
- Implementing DFS traversal to check connectivity.
- Reversing the graph and performing another DFS to verify strong connectivity.

The implemented approach is efficient, with a time complexity of $O(V + E)$, making it suitable for large graphs. Additionally, the program was developed without external libraries and follows a structured, modular approach for easy understanding and modification.

This project demonstrates the power of graph algorithms in solving real world problems related to connectivity in networks, social graphs, and web structures. Future improvements could include handling disconnected graph more efficiently or visualizing the graph structures for better interpretability.